

FieldFlow

Technical Documentation

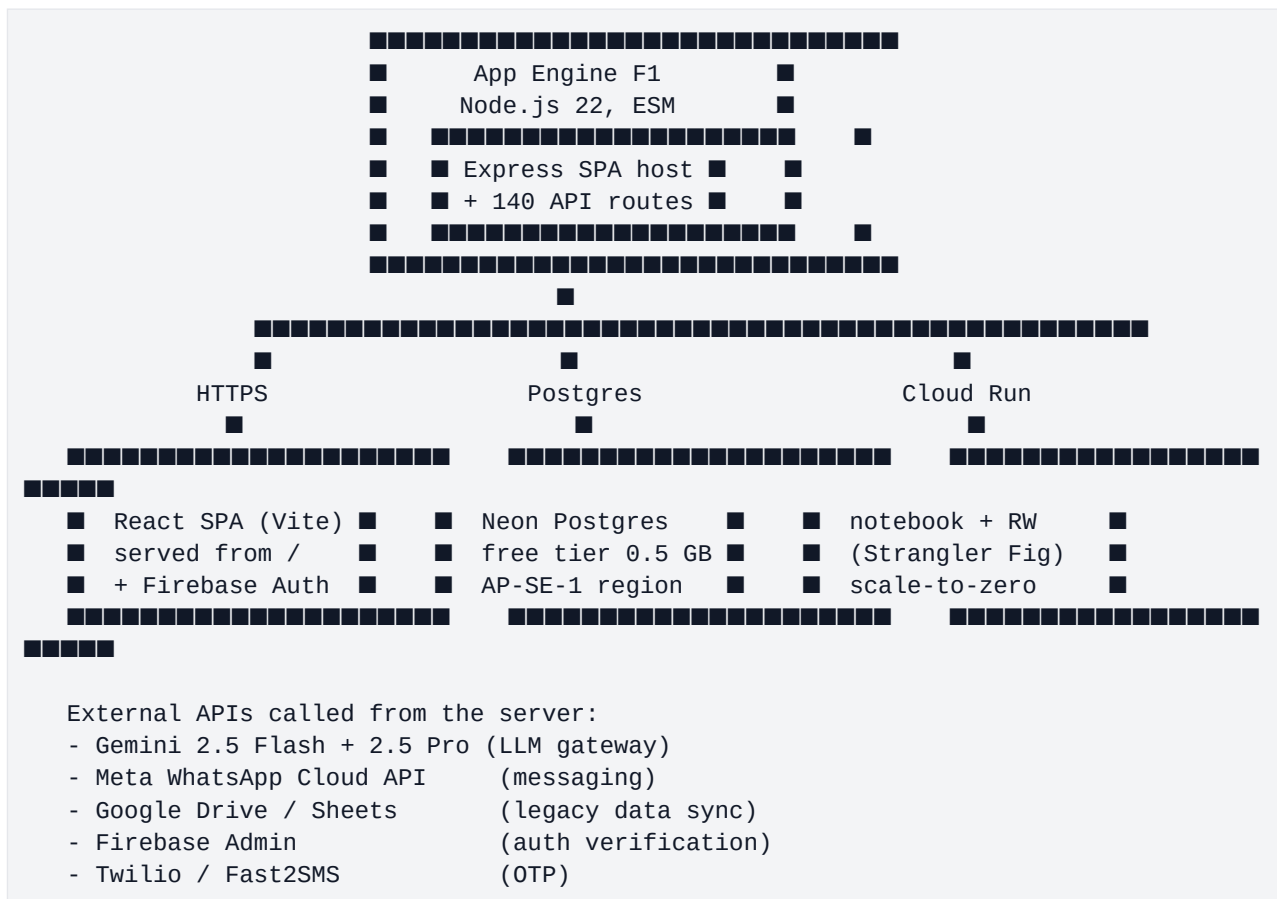
FieldFlow Technical Documentation

The engineering deep-dive: architecture, data model, AI infrastructure, WhatsApp internals, security posture, deployment, operations. Read this if you're joining the codebase, debugging production, or extending the platform.

Companion documents:

- [API.md](#) — endpoint reference
- [SDK.md](#) — integration patterns
- [Walkthrough.md](#) — end-user guide

1. Architecture overview



Why this shape

- **App Engine Standard** — free tier covers our load (1 always-on F1 instance, max 3). Auto-scales to zero on idle but we keep `min_instances: 1` to eliminate cold starts.
- **Neon Postgres** — replaced Cloud SQL (\$600/mo → \$0/mo). Free tier is 0.5 GB. Connection-string pooled access from Node-pg.

- **Cloud Run for notebook + report-writer** — heavyweight AI work runs on Cloud Run because it cold-starts faster than App Engine when scaled-to-zero. Strangler Fig pattern — the route proxy in App Engine forwards there if `NOTEBOOK_SERVICE_URL` is set, else it serves the AI work inline.
- **Single repo, single deploy** — monolith on purpose. We don't have the volume to justify microservices.

2. Tech stack

Runtime

- **Node.js 22 LTS** (App Engine runtime: `nodejs22`)
- **ES modules** (`"type": "module"` in `package.json`)
- **Top-level await** used in `server.js` for Secret Manager bootstrap

Server dependencies

Package	What it does
<code>express 4.22</code>	HTTP framework
<code>express-rate-limit</code>	Per-IP throttling on AI + auth endpoints
<code>helmet</code>	Security headers (CSP off — SPA needs inline)
<code>pg 8.13</code>	Postgres client (connection pool)
<code>firebase-admin</code>	ID token verification + custom claims
<code>bcrypt</code>	Password hashing
<code>xlsx</code>	Excel parsing & generation
<code>pdf-parse</code>	PDF text extraction (Notebook sources)
<code>docx, pptxgenjs</code>	Document/presentation generation
<code>papaparse</code>	CSV parsing
<code>dotenv</code>	Local dev env vars
<code>@google-cloud/secret-manager</code>	At-boot secret loading
<code>@google-cloud/bigquery</code>	Finance/attendance dashboard queries

Frontend dependencies

Package	What it does
<code>react 18 + react-dom</code>	UI framework
<code>vite 5</code>	Build + dev server
<code>tailwindcss 4 + @tailwindcss/vite</code>	Utility-first CSS
<code>@xyflow/react 12</code>	Visual flow canvas (WA flow builder)
<code>lucide-react</code>	Icon set
<code>firebase 11</code>	Client auth SDK
<code>xlsx</code>	Client-side parse of uploaded action plan files

Package	What it does
file-saver	Authenticated downloads (bypass <code><a href></code> token-skipping)

Build tools

- **TypeScript** strict mode on the frontend, JavaScript on the server (because we want zero compile step in prod)
- **Vite** for the SPA bundle. Server runs raw `node server.js`.

3. Repository layout

```

.
■■■ server.js                # Express entry – boots secrets, wires routes, starts
retention
■■■ app.yaml                 # App Engine deployment config
■■■ package.json
■■■ db/
■   ■■■ pool.js              # Singleton pg.Pool – Neon connection
■   ■■■ migrations/          # Versioned SQL – applied to fresh DBs
■       ■■■ 001_initial.sql
■       ■■■ 002_subscriptions.sql
■       ■■■ 003_rw_rls.sql
■       ■■■ 004_saved_reports.sql
■       ■■■ 005_whatsapp_submissions.sql
■       ■■■ 006_whatsapp_platform.sql
■       ■■■ 007_org_prompts.sql
■       ■■■ 008_ai_layer.sql
■       ■■■ 009_ai_gateway.sql
■■■ lib/
■   ■■■ auth.js              # Firebase Admin init + requireAuth middleware
■   ■■■ aiAgent.js           # initAiLayer() – runtime DDL for late-added tables
■   ■■■ crypto.js            # AES-256-GCM helpers for at-rest secret encryption
■   ■■■ correlationId.js     # Per-request CID middleware
■   ■■■ flowEngine.js        # WA flow execution engine (every node type)
■   ■■■ gemini.js            # Streaming SSE Gemini gateway
■   ■■■ nlp.js               # Non-streaming Gemini + intent detection
■   ■■■ memoryExtractor.js    # Memory candidate extraction from interactions
■   ■■■ promptStore.js       # Org-level prompt CRUD
■   ■■■ retention.js         # Daily cleanup of audit_log, wa_messages, DLQ
■   ■■■ secretManager.js     # Boot-time GCP Secret Manager loader
■   ■■■ sheets.js            # Google Sheets API helpers
■   ■■■ usageTracker.js      # Per-org token + cost tracking
■   ■■■ voiceTranscribe.js   # Gemini audio → text for WA voice notes
■   ■■■ whatsapp.js          # WA client (send + parse webhook events)
■■■ routes/                  # 19 route files, 140 endpoints total
■   ■■■ action-plan.routes.js
■   ■■■ admin.routes.js
■   ■■■ ai-settings.routes.js
■   ■■■ ai.routes.js
■   ■■■ auth.routes.js
■   ■■■ billing.routes.js
■   ■■■ finance.routes.js

```

```

■    ■■■■ flow-generator.routes.js
■    ■■■■ notebook.routes.js
■    ■■■■ org.routes.js
■    ■■■■ prompts.routes.js
■    ■■■■ quick-report.routes.js
■    ■■■■ report-writer.routes.js
■    ■■■■ reports.routes.js
■    ■■■■ saved-reports.routes.js
■    ■■■■ superadmin.routes.js
■    ■■■■ wa-platform.routes.js
■    ■■■■ whatsapp.routes.js      # Legacy Glific receiver
■    ■■■■ workers.routes.js
■■■■ scripts/                    # One-off ops scripts
■    ■■■■ generate_manual.py
■    ■■■■ md_to_pdf.py
■    ■■■■ migrate-to-neon.js
■    ■■■■ migrate-secrets-to-manager.sh
■    ■■■■ fix-neon-schema.mjs
■■■■ src/                        # Frontend
■    ■■■■ App.tsx
■    ■■■■ main.tsx
■    ■■■■ context/               # Auth, Org, Language, Report
■    ■■■■ components/
■    ■    ■■■■ dashboard/        # Tabs (Overview, Reports, Impact, ActionPlan, etc.)
■    ■    ■■■■ cards/           # ReportCard, ReportDetailModal
■    ■    ■■■■ whatsapp/        # WhatsApp module – FlowBuilder, Conversations, etc.
■    ■    ■■■■ notebook/       # NotebookLM-style RAG UI
■    ■    ■■■■ superadmin/      # Org/plan management UI
■    ■■■■ utils/                # apiFetch, authedDownload, docxExport
■    ■■■■ data/                 # actionPlanData.json (seed)
■    ■■■■ types/                # Shared TS types
■■■■ docs/
■    ■■■■ API.md
■    ■■■■ SDK.md
■    ■■■■ Walkthrough.md
■    ■■■■ Technical.md          # This file

```

4. Data model

Everything is in **Postgres on Neon**. There are 24 tables. All multi-tenant — `org_id` `UUID NOT NULL` REFERENCES `organizations(id)` on every business table.

4.1 Core multi-tenant tables

Table	Purpose
organizations	Tenant. Has slug, plan_id, metadata JSONB, subscription_status.
plans	Subscription plans (Starter, Pro, Enterprise).
users	Per-org users. Has firebase_uid, phone, role, manager_id, password (bcrypt for sheet-login), designation, project_ids.

4.2 Reporting

Table	Purpose
daily_reports	Field reports submitted by staff. Has report_date, state, location, project, area_of_intervention, beneficiaries, description, attachment_url, custom_data JSONB.
saved_reports	AI-generated reports. Has title, report_type, content_html, content_json, filters, instruction.
project_deliverables	Per-project indicator data. Used by Action Plan AND Excel import. Has project, indicator ("sn\
action_plans	Multi-project action-plan registry. Has project_key, name, year, start_month, locations TEXT[], activities JSONB (full structure).

4.3 WhatsApp tables

Table	Purpose
wa_config	Per-org WA settings. phone_number_id, access_token (encrypted), app_secret, webhook_secret, business_id.
wa_contacts	Each contact. Has wa_id (phone), name, fields JSONB (custom), opted_in, tags TEXT[] (collections), last_seen.
wa_flows	Flow definitions. Has name, trigger_keywords TEXT[], nodes JSONB, is_default, is_active, show_progress.
wa_sessions	Active conversation state. Has contact_id, flow_id, current_node, collected JSONB, status (active/handoff/completed/abandoned/expired), assigned_to (user UUID), expires_at.
wa_messages	Every inbound + outbound message. wa_message_id (Meta's wamid for dedup), direction, type, content JSONB, status (delivered/read/failed).
wa_broadcasts	Broadcast queue. name, message_body, recipient_filter JSONB, total_sent, total_failed.
wa_dead_letter	Inbound events whose processing failed. Auto-purged after 30 days.

4.4 AI infrastructure

Table	Purpose
org_prompts	Per-org overrides of system prompts. Replaces hardcoded prompts in lib/prompts/.
ai_memories	Long-term context for AI. Persona memories, organizational facts, project memories.
ai_interactions	Every AI call's prompt + response + cost. Used by lib/memoryExtractor.js.

Table	Purpose
ai_learning_candidates	Suggested prompt edits derived from interactions (admin reviews + approves).
ai_settings	Per-org AI tier preference, feature toggles.
ai_rule_versions	Versioned prompt-rule sets (rollback support).

4.5 Audit, retention, billing

Table	Purpose
audit_log	Every state-changing action. actor_uid, actor_name, action, target_type, target_id, diff JSONB.
usage_events	Per-org token + cost telemetry. model, input_tokens, output_tokens, cost_usd.
whatsapp_submissions	Legacy Glific webhook payload archive.

4.6 Report Writer

Table	Purpose
rw_profiles	Per-org RW profile (tone, language, style).
rw_glossary	Terms with preferred translations / definitions.
rw_drafts	Draft history per report.
rw_feedback	User-supplied feedback to drive learning.

4.7 Row-level security

`003_rw_rls.sql` enables RLS on report-writer tables. Server connects as `rw_service` for those queries; `fieldflow_app` for everything else.

5. Module deep-dives

5.1 Authentication (`lib/auth.js` + `routes/auth.routes.js`)

Two login paths:

Firestore phone OTP (modern):

1. Client sends OTP via Twilio/Fast2SMS via `POST /api/send-otp`
2. Client verifies via `POST /api/verify-otp` — gets a custom token
3. Client signs in to Firebase → gets ID token
4. Client calls `POST /api/auth/login` with ID token → server reads `firebase_uid` from token, looks up user, sets custom claims (orgId, role)

Sheet login (legacy):

1. Client sends { phone, password, orgSlug } to POST /api/auth/sheet-login
2. Server looks up org by slug, then user by phone in that org. Bcrypt-compare password.
3. If match, generate custom token, set claims, return.

requireAuth middleware verifies the ID token on every protected route. Populates req.user = { uid, orgId, role, name, phone }.

5.2 WhatsApp flow engine (lib/flowEngine.js)

The heart of the WA module. ~700 lines.

executeNode(node, collected, contact, waClient, to, pool, flow):

- Returns { wait, nextNodeId, updatedCollected }
- Dispatch table on node.type — 19 node types
- Each case may call waClient.sendText/sendList/sendButtons, mutate collected, call Gemini, etc.

runFlowFrom(nodes, startNodeId, collected, contact, waClient, to, maxSteps, pool, flow):

- Loops until it hits a wait node or next=null
- Cap maxSteps=20 prevents infinite loops

processReply(currentNode, incomingEvent, collected):

- Called when a new message arrives during an active session
- Routes the user's reply to the right variable, handles validation (number/phone/image)
- Returns newCollected

Variable scoping

- **Local** (session): collected JSONB. Cleared at session end.
- **Global** (contact): wa_contacts.fields JSONB. Written by update_contact node. Persists across flows.
- **Built-in:** contact.name, contact.wa_id.

Interpolation via interpolate(template, collected, contact) — replaces {{var}} and {{contact.field}}.

Special "control" variables in collected

- __last_text — most recent user text (any input)
- __handoff — true if human_handoff was reached
- __handoff_assignee — UUID of user assigned (if any)
- __validation_error — set during validation failure; route handler sends to user

- `__enter_flow` — set when `enter_flow` node fires; route handler starts the named flow
- `__recap_confirmed` — set after `recap_confirm` reply (YES vs anything else)

5.3 Inbound webhook handler (`routes/wa-platform.routes.js`)

```
POST /api/wa/webhook
  ■ Always res.sendStatus(200) immediately (Meta retries if not 200 within 20s)
  ■ Parse events via lib/whatsapp.js::parseWebhookEvents
  ■ For each event:
    try { handleWebhookEvent(event, pool, rawBody, sigHeader) }
    catch (err) { INSERT INTO wa_dead_letter ... }
```

`handleWebhookEvent`:

1. Look up `wa_config` by `phone_number_id`
2. Verify HMAC signature (fail-closed in production)
3. Decrypt `access_token`
4. If `event.type === 'audio'` → download from Meta API → transcribe via Gemini → set `event.text` to transcript
5. Upsert `wa_contacts` row
6. Insert `wa_messages` row (with `wa_message_id` unique constraint for dedup)
7. Look up active session:
 - **Session in handoff**: log message, return (bot stays paused)
 - **Session active**: `processReply(curNode, event, session.collected) → runFlowFrom` from `curNode.next` → update session
 - **No session**: keyword-match against active flows → if no match, run `detectIntent` NLU fallback → if still no match, send `generateFallbackResponse` warm reply

5.4 Action Plan internals

Per-cell data lives in `project_deliverables` keyed by (`org_id`, `project`, `indicator`). Indicator format: `"sn|location|month"` e.g. `"3|Khagaria|May"`.

Static seed comes from `src/data/actionPlanData.json` — bundled at build time, used as the bundled "Kosi Sahjivan 2026" plan. `initAiLayer` reads this JSON at startup and inserts an `action_plans` row for any org that has Kosi cell data but no plan registry row.

Aggregation (impact):

The `/api/action-plans/impact` endpoint joins `action_plans` × `project_deliverables` and computes per-project, per-category, per-location, per-month totals. Categories come from each activity's `category` field in the seed JSON. Two-pass aggregation:

1. Iterate every plan's `seedCells` (targets from the upload). Apply override from DB if present.
2. Iterate every DB cell not seen in pass 1 (user-added beyond seed).

`toNum` helper: Postgres `NUMERIC` serialises as `string` in node-pg. The frontend's `activityTotal` was concatenating strings (`0 + "2" + "2" = "022"`); now wraps every read through `toNum()`. Same pattern applies anywhere we sum numeric DB columns on the frontend.

5.5 AI infrastructure

Three layers:

Layer 1 — Direct Gemini calls (`lib/nlp.js::callGemini`, `lib/gemini.js`):

- Non-streaming for short structured outputs (intent detection, slot extraction)
- SSE streaming for long-form (`callGeminiNotebook`)
- All calls use `gemini-2.5-flash` by default, `gemini-2.5-pro` for high-quality report generation
- Hardened with prompt-injection guard in `call_llm` node

Layer 2 — Memory (`lib/memoryExtractor.js` + `ai_memories` table):

- After each `/ai/ask` interaction, extract candidate memories (persona, org facts, project info)
- Admin reviews + approves in `ai_learning_candidates`
- Approved memories flow back into the org's system prompt via `buildOrgContext`

Layer 3 — Rule versioning (`ai_rule_versions`):

- Each prompt edit creates a new version
- Active version is referenced via `org_prompts`
- Rollback supported via `POST /api/ai/learning/rollback`

5.6 Notebook (RAG)

Sources are passed per-request (no persistence layer yet — every chat call gets the full source list). Streaming SSE response.

`routes/notebook.routes.js`:

- `POST /api/notebook/upload` — decode base64 PDF, extract text via `pdf-parse`, cap to 200KB
- `POST /api/notebook/chat` — RAG: `{ sources, message, history }` → Gemini → stream tokens
- `POST /api/notebook/audio`, `tts`, `study-guide`, `slide-deck`, `imagen` — content generators

Sources have a `capSources` helper that trims to `{maxPerSource, maxSources}` to keep prompts under the Gemini context limit.

5.7 Report Writer (Strangler Fig)

`routes/report-writer.routes.js` is a proxy:

- If `RW_SERVICE_URL` env is set → forwards requests to Cloud Run

- Else → handles inline (fallback for local dev)

Cloud Run service is the same Node code but with `NODE_ENV=c loudrun` and its own connection pool. Lets us scale heavyweight AI work without bumping the main App Engine instance class.

6. Frontend architecture

6.1 Routing

Single SPA, no router library. The entire app is `App.tsx` rendering different tabs from `DashboardPage.tsx`. Tab state is a `useState` string union. URL doesn't reflect tab — partial limitation, doesn't matter for our use case.

6.2 Auth context

`src/context/AuthContext.tsx` wraps Firebase Auth SDK. Exposes `useAuthContext()` returning `{ user, signOut }` where `user` has `uid`, `role`, `orgId`, `name`, `phone`, `email`.

6.3 Org context

`src/context/OrgContext.tsx` fetches `/api/org/metadata` once on mount, caches the result. Exposes `useOrg()` returning the entire `OrgMetadata` (branding, modules, projects, etc.).

6.4 API access

`src/utils/apiFetch.ts` wraps `fetch`:

- Adds `Authorization: Bearer <token>`
- Auto-refreshes the token if a request returns 401
- Returns the raw `Response` so callers can choose `.json()` or `stream`

`src/utils/authedDownload.ts` — for file downloads from authed endpoints. Plain `<a href>` skips the Bearer header; `authedDownload(url)` fetches as a blob then triggers `file-saver`.

6.5 Key components

Component	Purpose
<code>DashboardPage.tsx</code>	Top-level layout. Sidebar + tab content. ~800 lines.
<code>ActionPlanTab.tsx</code>	Multi-project action plan grid + Status panel + Needs Attention + Quickfill + mobile cards. ~1500 lines.
<code>ImpactDashboard.tsx</code> + <code>ActionPlanImpactPanel.tsx</code>	Org-wide impact view with 3 sub-tabs.
<code>QuickReportTab.tsx</code>	AI-driven field-report form with voice + photos.
<code>WhatsAppPage.tsx</code>	Container for the WA module. 8 sub-tabs.
<code>FlowBuilderCanvas.tsx</code>	xyflow-based visual flow editor. 1100+ lines.

Component	Purpose
FlowSimulator.tsx	In-browser flow runner.
ConversationsTab.tsx	Live inbox with AI-suggested replies + Mine filter + handoff controls.
ContactsList.tsx	Paginated contact list with multi-select + bulk add-to-collection.
CollectionsTab.tsx	Tag-based contact groups.
BroadcastPanel.tsx	Bulk message composer with audience picker.
AnalyticsTab.tsx	Message volume, flow performance, NLU health, broadcast deliverability.
UploadActionPlanModal.tsx	Drag-drop XLSX parser (lazy-loads xlsx).
NotebookPage.tsx	Source list + chat + audio + study-guide + slide-deck UIs.

6.6 State management

No Redux/Zustand. Each component manages its own state with `useState` + lifts state via props for cross-component coordination. Global concerns (auth, org) use Context. Persisted state (`activePlanKey`, `compactMode`, etc.) uses `localStorage` with simple `JSON.parse/stringify`.

6.7 Build

- `npm run build → tsc -b && vite build`
- Output in `dist/`. `server.js` serves it as static files with `serveIndex` fallback for SPA routes.
- Heavy chunks code-split:
- `xlsx` chunk (143 KB gzipped) — lazy-loaded only when upload modal opens
- `pptxgen.es` (127 KB) — lazy-loaded for slide deck generation

Main bundle: ~476 KB gzipped (large because of React 18 + xyflow + many tabs).

7. Security

7.1 Authentication

- All `/api/*` except a few public endpoints require `Authorization: Bearer <firebase-id-token>`
- `requireAuth` middleware in `lib/auth.js` verifies tokens via Firebase Admin SDK
- Custom claims (`orgId`, `role`, `name`) baked into token; no DB lookup per-request

7.2 Authorization

- Role hierarchy: `superadmin > admin > manager > employee`
- Role checks per-route: `if (!['admin', 'superadmin'].includes(req.user.role)) return res.status(403)...`

- `requireEditor` helper in action-plan routes
- Frontend gates tabs via `canImpact = isAdmin || isManager`

7.3 Multi-tenant isolation

Every business table has `org_id NOT NULL`. Every query filters by `org_id = $1` using `req.user.orgId` (cannot be forged — Firebase Admin verifies the JWT signature).

Cross-org tampering checks audited via `grep`:

- `UPDATE` queries always include `AND org_id = $`
- `DELETE` queries always include `AND org_id = $`
- One exception found + fixed during security pass: `POST /wa/flows/:id/publish` had a missing org filter on the `UPDATE`.

7.4 Webhook signature verification

Meta WA webhook signs every payload with HMAC-SHA256 using your App Secret. We verify in `lib/whatsapp.js::verifyWebhookSignature`. **Fail-closed in production** — if no `app_secret` is configured (per-org or `WA_APP_SECRET` env), inbound events are refused.

7.5 Secrets management

- `lib/secretManager.js` loads from GCP Secret Manager at boot
- Per-secret precedence: `process.env.X` (from `app.yaml` or shell) > Secret Manager
- `lib/crypto.js` provides AES-256-GCM for at-rest encryption of WA access tokens — stored as `enc:v1:<iv>:<ciphertext>` in `wa_config.access_token`
- Legacy plaintext tokens still read correctly; re-encrypted on next write

7.6 Rate limiting

Scope	Limit
AI endpoints	30 req/min
<code>/api/prewarm</code>	10 req/min
Login + verify-otp	8 req/min
Send-OTP	3 req/min

Via `express-rate-limit` middleware. Returns 429 + Retry-After header.

7.7 Body limits

- Webhook routes: 256kb
- File upload (notebook): 20mb
- AI data routes: 5mb
- Default: 512kb

Errors return 413 instead of crashing.

7.8 Logging hygiene

- Inbound WA messages logged with masked phone (`***1234`) + 40-char preview only
- Production error responses include only `cid` (correlation ID), no `e.message`
- Audit log diff stores `notes_changed: true` instead of before/after note content (PII protection)

7.9 Crash-proofing

- `process.on('unhandledRejection') + process.on('uncaughtException')` — log + continue, never `process.exit()`
- Global Express error middleware — catches anything escaping per-route try/catch
- Graceful shutdown on SIGTERM (App Engine sends this ~30s before kill) — drains in-flight requests up to 25s
- Request timeout 60s (defends against slowloris)

7.10 Retention (PII auto-purge)

Via `lib/retention.js` — runs once 30s after boot, then every 24h:

- `wa_dead_letter` > 30 days → delete
- `wa_messages` > 180 days → delete
- `audit_log` > 365 days → delete
- `wa_sessions` (completed/abandoned) > 30 days → delete
- `action_plans` (soft-deleted) > 30 days → hard delete

7.11 Known remaining risks

- `xlsx` package has known prototype-pollution issues — no fix available on npm. Mitigated by client-side-only parsing (server never receives XLSX bytes).
- `esbuild` dev-server SSRF — only affects local `npm run dev`, not production.
- `uuid` transitive vuln in `firebase-admin` — waiting on upstream fix.

8. Deployment

8.1 App Engine

- `app.yaml` declares `runtime: nodejs22`, `instance_class: F1`, `min_instances: 1`, `max_instances: 3`
- `gcloud app deploy app.yaml --quiet --project chatbot-492915` deploys

- Cold-start avoided by `min_instances: 1`. Within free tier budget.

8.2 Cloud Run microservices

- `fieldflow-notebook` and `fieldflow-report-writer` services
- Configured via `NOTEBOOK_SERVICE_URL` and `RW_SERVICE_URL` env vars in `app.yaml`
- Cost: ~\$0 because scale-to-zero
- Same codebase; deployed independently via `gcloud run deploy`

8.3 Environment variables (`app.yaml`)

Sensitive ones now migrated to Secret Manager:

- `GEMINI_API_KEY`, `DATABASE_URL`, `FIREBASE_ADMIN_SDK_JSON`, `GOOGLE_SERVICE_ACCOUNT_JSON`, `WA_WEBHOOK_VERIFY_TOKEN`, `ENCRYPTION_KEY`, `WA_APP_SECRET`

Non-sensitive (still in `app.yaml`):

- `NODE_ENV=production`, `FIREBASE_PROJECT_ID`, `VITE_FIREBASE_*` (frontend SDK config — public by design), `NOTEBOOK_SERVICE_URL`, `RW_SERVICE_URL`

8.4 Database

- Neon free tier (0.5 GB, AP-SE-1 Singapore region)
- Connection via pooled `DATABASE_URL`
- `db/migrations/*.sql` applied manually once on bootstrap; thereafter runtime DDL via `lib/aiAgent.js::initAiLayer` adds new tables/columns idempotently (`CREATE TABLE IF NOT EXISTS, ADD COLUMN IF NOT EXISTS`).

8.5 First-time setup checklist

1. Create GCP project, enable App Engine + Cloud Run + Secret Manager + BigQuery APIs
2. Create a Neon account, generate `DATABASE_URL`, run `db/migrations/*.sql` once
3. Create Firebase project, download admin SDK JSON
4. Get Gemini API key from AI Studio
5. Set up WhatsApp Business Account in Meta, get `phone_number_id` + access token + `app_secret`
6. Populate Secret Manager via `scripts/migrate-secrets-to-manager.sh` OR put values directly into `app.yaml` (less secure)
7. Set Cloud Run notebook + RW services if you want them separate (optional)
8. `gcloud app deploy` — App Engine builds the React bundle + starts Node

8.6 Continuous deployment

There's no CI/CD set up. Deploys are manual:

```
gcloud auth login
gcloud app deploy D:\chatbot\app.yaml --quiet --project chatbot-492915
```

Future: GitHub Actions on push to `main` would deploy automatically.

9. Observability

9.1 Correlation IDs

`lib/correlationId.js` middleware assigns a short ID to every request. Logged in every error message; returned to the client as `X-Correlation-ID` header and embedded in error JSON as `cid`.

9.2 Server logs

- `console.log/warn/error` → Google Cloud Logging (auto-collected by App Engine)
- Query via `gcloud app logs read --service=default --project=chatbot-492915 --limit=50`
- Or filter by CID: `gcloud logging read 'jsonPayload.cid="abc12345"'`

9.3 Usage tracking

`lib/usageTracker.js` writes a row to `usage_events` for every Gemini call:

- `org_id`, `model`, `input_tokens`, `output_tokens`, `cost_usd`, `created_at`
- Aggregated in `routes/billing.routes.js` for the superadmin billing dashboard

9.4 Dead-letter queue

`wa_dead_letter` table captures every webhook event whose processing failed. Admin can list (GET `/api/wa/dead-letter`) and retry (POST `/api/wa/dead-letter/:id/retry`).

9.5 What we don't have (yet)

- Distributed tracing (no OpenTelemetry)
 - Real-time error tracking (no Sentry)
 - Performance APM (no Cloud Profiler enabled)
 - Synthetic monitoring (no probe pinging `/whatsapp/health`)
-

10. Operations

10.1 Common runbook items

"Site is slow"

1. `gcloud app describe --service=default` — check current instance count
2. Check `wa_dead_letter` size — if growing, processing errors
3. Check Neon dashboard for slow queries

"WhatsApp not delivering messages"

1. `SELECT * FROM wa_config WHERE org_id=...` — config exists?
2. Try `POST /api/wa/contacts/:id/send` — does it return success?
3. Check Meta Business Manager — webhook still verified?
4. Tail App Engine logs during a real inbound message

"Action Plan numbers look wrong"

1. Check `project_deliverables` for the cells in question
2. NUMERIC columns return as strings — verify frontend uses `toNum()`
3. Check the active plan's `activities JSONB` for seed targets

"Token says invalid"

1. Firebase ID tokens expire after 1 hour. Force-refresh: `auth.currentUser.getIdToken(true)`
2. Custom claims need re-sign-in to take effect. Sign out → sign in.

10.2 Database backup

Currently: Neon free tier offers ~7 days of point-in-time recovery, no scheduled backups.

Recommended: add a nightly `pg_dump` to GCS bucket via Cloud Scheduler + small Cloud Function. Not yet set up.

10.3 Migrations

New schema changes go in `db/migrations/NNN_*.sql`. Idempotent (`CREATE TABLE IF NOT EXISTS`, etc.). For new orgs use the migration files directly. For existing prod, runtime DDL in `initAiLayer` applies safe column additions and index creation.

10.4 Rolling forward / back

- Forward: `gcloud app deploy` — App Engine routes 100% traffic to new version atomically
- Back: `gcloud app services set-traffic default --splits=<old-version>=1`
- All deployed versions stay around (purgeable via `gcloud app versions delete`)

11. Performance

11.1 Cold start mitigation

- `min_instances: 1` keeps one warm
- `/api/_ah/warmup` pre-fetches DB pool

11.2 Bundle size

- Main bundle: 476 KB gzipped — large but acceptable for an authed B2B SaaS dashboard
- `xlsx + pptxgen` chunks lazy-loaded via `dynamic import()`

11.3 Database

- All hot queries indexed (`org_id`-first composite indexes)
- `wa_messages.wa_message_id` non-partial unique index for ON CONFLICT dedup
- `project_deliverables` uses (`org_id, project, indicator`) upsert key
- Bulk operations chunked at 500 rows per multi-VALUES INSERT to avoid PG's 65535 param limit

11.4 SSE streaming

AI endpoints stream tokens via SSE. Client renders incrementally — first paint at ~500ms even for 30-second generations.

11.5 Caching

- `lib/nlp.js` has an 8-min in-process LRU cache (40 entries) for `analyzeFieldData` results — same report batch + same instruction returns instantly second time
- No HTTP cache headers on API responses (everything is per-org dynamic)
- No CDN — App Engine serves static SPA directly (fine for our traffic)

12. Cost model

Monthly (at current usage):

Service	Cost
Neon Postgres (free tier 0.5 GB)	■0
App Engine F1 × 1 (within free tier 28 instance-hours/day)	■0
Cloud Run notebook + RW (scale-to-zero)	■0
GCS buckets (<5 GB)	~■5
Secret Manager (≤6 active secrets)	■0
Total infrastructure	~■0/month

Usage-based (varies):

- Gemini 2.5 Flash: \$0.075 / 1M input, \$0.30 / 1M output → ~\$0.50-2 per AI report
- Gemini 2.5 Pro (when used): \$1.25 / 1M input, \$5.00 / 1M output
- WhatsApp Cloud API: free for first 1000 conversations/month, ~\$0.50/message after
- Twilio SMS OTP: ~\$0.20/message

Tracked per-org in `usage_events`. Visible in superadmin billing dashboard.

13. Migration history

The project has gone through one major migration:

Cloud SQL → Neon (June 2026):

- Old: PostgreSQL 16 on Cloud SQL (db-custom-8-32768, ~\$8000/month)
- New: Neon free tier (0.5 GB Postgres)
- Driven by: zero-cost goal
- Migration via `scripts/migrate-to-neon.js` — copied schema + data preserving everything
- Hidden bug discovered during migration: `wa_messages.wa_message_id` unique constraint was lost. Re-added as non-partial unique index in `initAiLayer`.
- Other instances also deleted: MySQL `fieldflow-db` (unused), PostgreSQL `fieldflow-pg`.

Future migrations of note:

- (None planned. Neon will scale to paid tier if traffic justifies.)
-

14. Roadmap

Documented in the plan file at `~/.claude/plans/eager-beaming-stallman.md`. High-level:

Closed today (recent work)

- Multi-project Action Plan with Excel upload
- Impact dashboard with category + activity breakdown
- WhatsApp module hardening (handoff, AI replies, suggested replies, voice transcription, collections, broadcast targeting)
- Action Plan UX cleanup (header split, Status panel merge, sticky-column fix, hierarchy tightening)
- 23-issue security audit + fixes
- Secret Manager migration
- Documentation (API, SDK, Walkthrough, Technical)

Strategic bets still on the table

1. **AI Flow Generator v2** — template gallery + iterate-on-flow mode
2. **Closed-loop integration** — WA survey completions → write to `daily_reports` automatically (the moat against Glific)
3. **RAG over WhatsApp** — `ask_kb` node that queries Notebook PDFs and returns cited answers
4. **Government compliance exports** — PMMSY/PMKSY/NRLM Excel formats with PII guard
5. **Localization** — make every flow text field language-aware (Hindi/Marathi/Bengali etc.)

Operational follow-ups

- Nightly `pg_dump` to GCS bucket (DB backup)
- OpenTelemetry / Sentry integration
- CI/CD via GitHub Actions
- Synthetic monitoring of `/whatsapp/health`

15. Glossary

Term	Meaning
CID	Correlation ID — short string identifying a single request. In every log line and error response.
DLQ	Dead-letter queue. <code>wa_dead_letter</code> table.
Flow	A WhatsApp conversation script (nodes + edges).
HSM	Highly Structured Message — Meta's pre-approved template messages, required after 24h session window.
NLU	Natural Language Understanding — using AI to match free-form contact input to a flow.
PII	Personally Identifiable Information.
PWA	Progressive Web App. We're not technically one (no SW), but the dashboard works on mobile.
RAG	Retrieval-Augmented Generation. The Notebook tab pattern.
RBAC	Role-Based Access Control.
RLS	Row-Level Security (Postgres feature).
SSE	Server-Sent Events. Used for streaming AI responses.
Strangler Fig	Pattern where parts of an old service are gradually replaced by new ones behind a proxy. Used for notebook + RW microservices.
ToC	Theory of Change — NGO impact framework.
WABA	WhatsApp Business Account.
wamid	Meta's globally-unique message ID.

16. Appendices

Appendix A — Full table list

24 tables across all migrations + runtime DDL:

```
ai_interactions
ai_learning_candidates
ai_memories
ai_rule_versions
ai_settings
action_plans
audit_log
daily_reports
organizations
org_prompts
plans
project_deliverables
rw_drafts
rw_feedback
rw_glossary
rw_profiles
saved_reports
usage_events
users
wa_broadcasts
wa_config
wa_contacts
wa_dead_letter
wa_flows
wa_messages
wa_sessions
whatsapp_submissions
```

Appendix B — Env var checklist

Required (server can boot without these but specific features will degrade):

- `DATABASE_URL` — Postgres connection string
- `FIREBASE_PROJECT_ID`
- `FIREBASE_ADMIN_SDK_JSON` — service account
- `GEMINI_API_KEY`
- `GOOGLE_SERVICE_ACCOUNT_JSON` — for Sheets/Drive
- `WA_WEBHOOK_VERIFY_TOKEN`

Optional (with fallbacks):

- `ENCRYPTION_KEY` — falls back to deterministic key from project ID
- `WA_APP_SECRET` — falls back to per-org `wa_config.app_secret`

- `NOTEBOOK_SERVICE_URL` — if set, proxy notebook routes to Cloud Run
- `RW_SERVICE_URL` — same for Report Writer
- `TWILIO_SID`, `TWILIO_TOKEN`, `TWILIO_PHONE`, `FAST2SMS_KEY` — at least one for OTP

Frontend env (baked into bundle at build time):

- `VITE_FIREBASE_API_KEY`
- `VITE_FIREBASE_AUTH_DOMAIN`
- `VITE_FIREBASE_PROJECT_ID`
- `VITE_FIREBASE_STORAGE_BUCKET`
- `VITE_FIREBASE_MESSAGING_SENDER_ID`
- `VITE_FIREBASE_APP_ID`

Appendix C — Useful queries

Find a user across all orgs:

```
SELECT u.id, u.name, u.phone, u.role, o.slug, o.name AS org_name
FROM users u JOIN organizations o ON o.id = u.org_id
WHERE u.phone LIKE '%9876%' OR u.name ILIKE '%priya%';
```

Active sessions right now:

```
SELECT s.id, c.wa_id, c.name, f.name AS flow, s.current_node, s.status, s.updated_at
FROM wa_sessions s
JOIN wa_contacts c ON c.id = s.contact_id
JOIN wa_flows f ON f.id = s.flow_id
WHERE s.status IN ('active', 'handoff')
ORDER BY s.updated_at DESC;
```

Per-org cost this month:

```
SELECT org_id, model, SUM(cost_usd) AS total_usd
FROM usage_events
WHERE created_at >= date_trunc('month', NOW())
GROUP BY org_id, model
ORDER BY total_usd DESC;
```

Action Plan cells behind schedule:

```
SELECT pd.org_id, pd.project, pd.indicator, pd.planned, pd.achieved,
       ROUND(100.0 * pd.achieved / pd.planned, 1) AS pct
FROM project_deliverables pd
WHERE pd.data_type = 'action_plan'
      AND pd.achieved IS NOT NULL
      AND pd.planned > 0
      AND (pd.achieved::numeric / pd.planned::numeric) < 0.5
ORDER BY pct;
```

Appendix D — Common commands

```
# Deploy
gcloud app deploy app.yaml --quiet --project chatbot-492915

# Tail logs
gcloud app logs read --service=default --project chatbot-492915 --limit=50

# Query Neon directly
psql "$DATABASE_URL"

# Build + test locally
npm install
npm run build
npm start

# Generate doc PDFs
python scripts/md_to_pdf.py docs/API.md docs/API.pdf
python scripts/md_to_pdf.py docs/SDK.md docs/SDK.pdf
python scripts/md_to_pdf.py docs/Walkthrough.md docs/Walkthrough.pdf
python scripts/md_to_pdf.py docs/Technical.md docs/Technical.pdf
```

17. Contributing

The project is currently maintained by a small team. Branch naming, PR templates, CODE_OF_CONDUCT, etc. are not yet formalised.

House rules:

- Don't add features beyond what's been agreed
- Don't add error handling for impossible cases — trust internal code
- Don't write comments that explain what the code does — only why it's non-obvious
- Don't introduce a new library unless it's the only sensible path (we have 30+ already)
- Prefer modifying an existing file to creating a new one

House style:

- Server: JavaScript (ESM), no TypeScript on the server side
- Frontend: TypeScript strict mode
- Naming: snake_case in DB, camelCase in JS/TS, PascalCase for React components
- 2-space indentation everywhere
- No semicolons in the frontend (mostly); semicolons fine on the server
- Tailwind for styling. Avoid CSS files except global resets.

End of document. ~16 chapters, ~24 tables documented, ~140 endpoints covered, 4 PDFs total in this doc set.